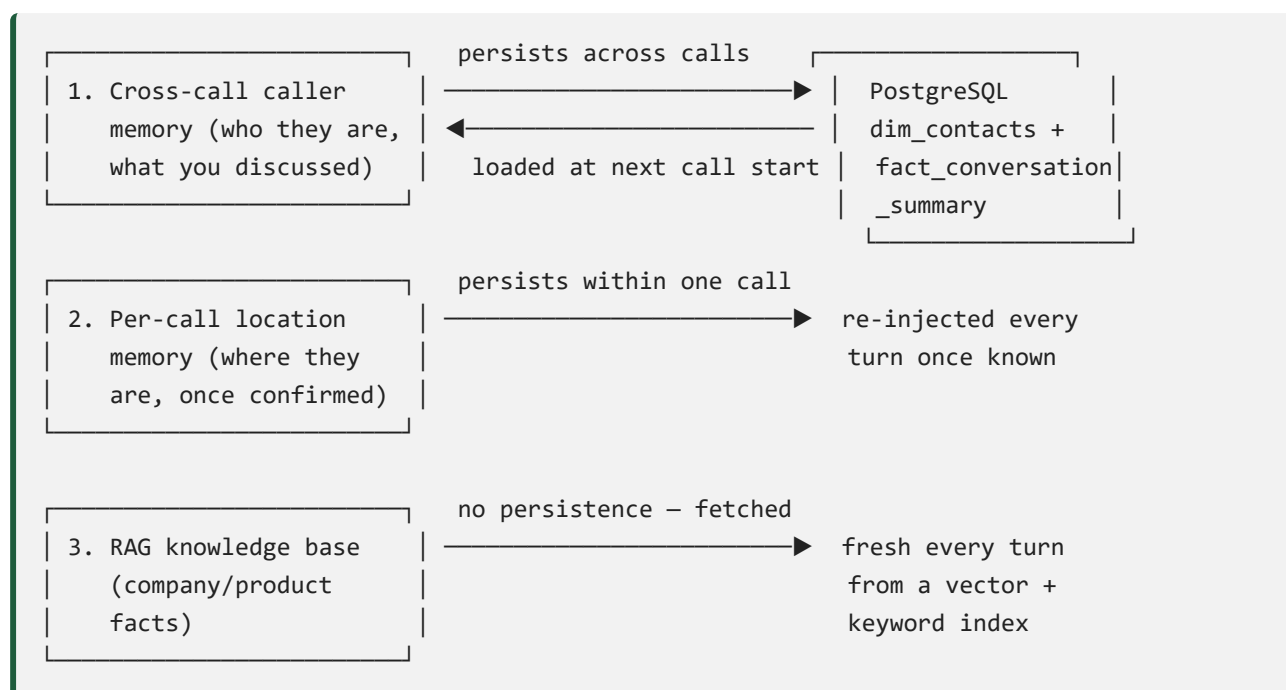


Context & Memory — Implementation Report

How Farm Vaidya's voice bot remembers things — across calls, within a call, and from its own knowledge base — what was built, the tools behind each piece, and a real bug found and fixed in this system on 2026-08-11.

1. Three separate "memory" systems, not one

"Context memory" isn't a single mechanism here — it's three independent systems, each solving a different forgetting problem, all converging on the same place: messages injected into the LLM's conversation context (`pipecat.processors.aggregators.llm_context.LLMContext`).



2. Cross-call caller memory

Problem it solves: a caller who called last week shouldn't have to re-explain who they are or what they were asking about.

Storage — two Postgres tables (`db/schema.sql`):- `dim_contacts` — one row per phone number: name, first/last seen, call count, and (once captured) their confirmed farming location. -

`fact_conversation_summary` — one row **per call**, but each row is a **cumulative** summary: call 11's saved summary already folds in everything from calls 1–10, so loading a caller's context only ever needs their single latest row, never the full history. This is a deliberate design choice, not an oversight — see `bot_processors/calls/caller_memory.py` 's own module docstring.

Files: - `bot_processors/calls/caller_db.py` — `dim_contacts` reads/writes (`record_call` , `get_location` , `save_location` , `update_contact_name`).- `bot_processors/calls/caller_memory.py` — `fact_conversation_summary` reads/writes (`load_latest_summary` , `save_summary`) and the two greeting builders. - `bot_processors/calls/caller_summarizer.py` — the actual summarization logic: folds the previous cumulative summary + this call's transcript into one updated summary via an LLM call.

Flow, start to end of one call (`Bot.py` 's `_greet_and_inject_memory` and `on_client_disconnected`):

1. **Call connects.** Four independent DB operations run concurrently via `asyncio.gather` (not sequentially — the LLM greeting call is already the slow part, no reason to pay four round-trips serially first):
`record_call()` (bumps call count), `start_call()` (session bookkeeping), `load_latest_summary()`, `get_location()`.
2. **If a summary exists**, the caller is "known." Two greeting modes (`RETURNING_CALLER_GREETING_MODE` in `.env`):
3. `template` — a fixed Telugu sentence with the name filled in, zero extra latency.
4. `llm` — one extra out-of-band LLM call (`caller_summarizer.build_llm_greeting`) generates a natural line referencing their specific last topic, falling back to the template on any failure.
5. **The summary text itself is injected as a system -role message** (`note_from_summary`) — explicitly marked "for your context only — do not read this out loud verbatim."
6. **If a confirmed location exists**, a second system message states it and tells the LLM never to ask for district/state/village/pincode again this call.
7. **During the call**, every turn (see §4 below) re-injects a short location reminder if one is known — not a one-time thing.
8. **Call ends** (`on_client_disconnected` → `_finalize_call` → `summarize_and_save_call`): the full transcript (flattened to plain `Caller: ... / Bot: ...` text, not passed as live chat turns — passing it as real turns makes the model want to *continue* the conversation rather than analyze it) plus the previous cumulative summary go to one more LLM call, producing an updated `Name / LastTopic / Summary` block, parsed and saved as a **new row** under that phone number.

Real production issue this system had: the summarization LLM occasionally degenerated into repeating the same sentence 3+ times in a row (a known failure mode for some models under certain prompts) — `_dedupe_repeated_runs()` collapses any such run before saving. Also, the model sometimes glossed a Telugu name with an English parenthetical (e.g. "శ్రీధర్ (Kavitha)"), which the template greeting would then read aloud verbatim, saying the name twice in two languages — `_PAREN_GLOSS_RE` strips this as a second line of defense on top of the prompt instruction.

3. Per-call location memory — and the bug it had

Problem it solves: once a caller has stated their location earlier in *this* call, every later turn needs to remember it too, not just the turn right after they said it.

Why it's separate from a one-time injection: a single injection right when the location becomes known was tried first and wasn't reliable enough — confirmed live that the LLM still asked "which district?" for a price question just one turn later. `Bot.py` 's `on_user_turn_started` handler re-adds a short reminder **every turn**, not once.

The bug (found and fixed 2026-08-11)

The reminder was added fresh every turn via `context.add_message({"role": "system", ...})` — but never removed the **previous** turn's copy first. Two things compounded:

1. **Gemini/Vertex has no inline system-role turn** — mid-conversation `system` / `developer` -role messages silently collapse to `user` role (`gemini_adapter.py` 's `_from_standard_message` , a documented,

intentional part of the adapter, not a bug in Pipecat). So every one of these reminders became a **fake message from the caller**.

- 2. Without removing the old one, this **piled up one near-duplicate fake caller turn per turn** for the rest of the call.

Several stacked, near-identical fake "caller" messages in a row was enough to confuse the model into **literally reading one back out loud** as its own reply — confirmed live in a real call
(`call_919949070894_3032625a.log`): the bot spoke *"Use it directly for get_price/get_price_all_markets/get_weather whenever they"* — a verbatim fragment of the internal reminder — to an actual caller.

Fix: before adding each turn's reminder, find and remove any previous turn's copy first (matching an exact prefix constant, `_LOCATION_REMINDER_PREFIX`). This is the same fix `bot_processors/rag/rag.py` 's `RAGInjector` had already needed for its own per-turn injection (§4) — applied to `Bot.py` using the identical pattern.

4. RAG (company/product knowledge)

Problem it solves: questions about the company or its products that aren't covered by a live tool call (price/weather) need grounding from an actual knowledge base, not the model's own guesses.

Not persisted — this is fetched fresh every turn, nothing saved between calls or even between turns within a call (except that the *previous* turn's injection is explicitly removed before adding the new one — see below).

Pipeline (`bot_processors/rag/rag.py` , backed by `chonkie_rag/`) : - **Chunking:** `chonkie` (a text-chunking library). - **Dense embeddings:** Google Vertex AI's `text-multilingual-embedding-002` model (768-dim vectors). - **Vector storage/search:** `Qdrant` (`chonkie.QdrantHandshake`). - **Keyword search:** `rank_bm25` 's `BM25Okapi` , run alongside the dense search. - **Fusion:** Reciprocal Rank Fusion (RRF, Cormack et al. 2009) combines the dense and keyword result rankings into one.

Flow, per turn: 1. `RAGInjector` sits before the LLM context aggregator, watching each user turn (buffered across STT fragments — one utterance can arrive as several `TranscriptionFrame` s, all searched against the *accumulated* turn text, not each fragment in isolation, so an early fragment's correct match doesn't get wiped by a later fragment with no keyword hits). 2. `classify_intent()` first — a price/weather/location question skips RAG search entirely (those are answered by live tools, not the knowledge base). 3. Otherwise, hybrid search runs; results (or an explicit "none found" marker with instructions on how to proceed anyway) are injected as a `system` -role message — same Gemini system→user collapse as §3, but this system was built with the fix already in place: **the previous turn's injected passages are found and removed before adding the new one**, so passages don't pile up or leak into unrelated later answers.

5. Tools and libraries, by layer

Layer	Tool/Library	Role
Cross-call memory	PostgreSQL + <code>asyncpg</code>	Where names, locations, and summaries persist
Cross-call memory	Google Vertex AI (Gemini, via <code>run_inference</code>)	The out-of-band summarization + greeting-

Layer	Tool/Library	Role
		generation calls
Cross-call/location	<code>pipecat.processors.aggregators.llm_context.LLMContext</code>	The in-memory conversation context both systems inject into
Location memory	Python <code>re</code>	Prefix-matching to find/remove a prior turn's injected message
RAG	<code>chonkie</code>	Text chunking
RAG	Google Vertex AI <code>text-multilingual-embedding-002</code>	Dense embeddings
RAG	Qdrant	Vector search
RAG	<code>rank_bm25</code> (BM25Okapi)	Keyword search, fused with the vector results (RRF)
RAG	<code>httpx</code> , <code>numpy</code>	HTTP calls, vector math
All layers	<code>loguru</code>	Logging throughout

6. Known characteristics, stated plainly

- **The cumulative-summary design means old detail gets condensed or dropped**, not preserved verbatim forever — this is intentional (a 10-call history compressed to 300 words has to lose something), but worth knowing if you ever need an exact quote from call #2's transcript — that's only in `fact_conversations` (the raw per-message log), not the summary.
- **RAG's "none found" marker is not itself an answer** — it explicitly instructs the model to still use live tools (price/weather) if it has enough information, rather than treating an empty knowledge-base result as "nothing available."
- **The §3 bug's root cause (Gemini's system→user collapse) is structural to the adapter**, not something patched away — any *future* per-turn injection added to this codebase needs the same "remove the previous copy first" pattern, or it will hit the identical failure mode.